Fair but hard: certified unique-solution puzzles generated by language models

Sara Behnamian
Department of Biology, Lund University, Sweden
sara.behnamian@biol.lu.se

Abstract

Language models can write logic puzzles, but cannot check that a puzzle has exactly one answer, and a solver can only check a puzzle once it has been translated into constraints. We build a generate–encode–certify pipeline for four families (zebra puzzles in English, Sudoku, Killer Sudoku and Futoshiki) in which two independent model calls encode the text, a SAT solver proves the two encodings equivalent and the puzzle unique, and a ladder of five nested deduction engines grades how much reasoning each puzzle needs. Across 299 zebra puzzles, encoders made no error on template text and erred on 0–8% of puzzles with harder wording or model-written paraphrases. Dual encoding caught every error that only one encoder made but missed errors both made; the uniqueness check caught those, and no wrong encoding passed both checks. Puzzles written by a model were nearly always valid (98 of 100) but easy and redundant, and some Sudoku copied a well-known published puzzle. The ladder correlates strongly with solver effort and explanation size in every family, but three Claude models solved almost all fair puzzles regardless of level; failures, many of them unanswered items, appeared mainly for the smallest model on 9×9 Sudoku. Evolving fair zebra puzzles against the smallest model found no puzzle it failed. All 240 model calls were run as isolated agent calls rather than through the API, at reduced scale. We release the pipeline, a benchmark of certified puzzles with levels and minimal explanations, and every intermediate result.

1 Introduction

A logic puzzle is a promise: there is exactly one answer, and it can be reached by reasoning rather than guessing. Generating puzzles that keep this promise is an old problem in procedural content generation [2, 24, 26]. For grid puzzles it is usually solved with a constraint solver: Sudoku, for example, is a small exact-cover problem that a SAT or constraint solver settles in milliseconds [14, 25, 28], and deciding whether a puzzle has a second solution is the standard way to certify uniqueness, even though the problem is hard in general [30]. Minimal puzzles and rated difficulty are well studied for Sudoku [16, 19], and step-wise explanations of why each cell value is forced can be extracted from minimal unsatisfiable subsets [1].

Large language models (LLMs) are now asked both to write such puzzles and to solve them. Grid-logic puzzles have become a standard probe of LLM reasoning [13, 27]. Two problems follow. First, a model that writes a puzzle in natural language cannot check that the puzzle has one answer, or that the answer it states is the right one. Second, any fixed benchmark built from such puzzles can leak into training data [7, 22]; dynamic benchmarks are one response [10].

Pairing an LLM with a solver addresses the first problem, and has been explored for logical reasoning [17, 18, 31] and for formalising mathematics [29]. But a certificate is only as good as the translation behind it. If the formal encoding misreads "directly to the left of" as "somewhere to the left of", the solver certifies the wrong puzzle. We therefore treat the translation itself as something to verify.

This paper builds and tests a generate–encode–certify loop for four puzzle families (zebra puzzles in English, Sudoku, Killer Sudoku and Futoshiki) and uses it for three purposes.

1. *Faithfulness.* Two independent LLM calls encode the same puzzle text into a small constraint language; a solver proves the two encodings equivalent or returns a disagreement. We measure how often single encodings are wrong, what kind of errors occur, and how many the dual check catches (E1). We also measure how often LLM-written puzzles are valid without the loop (E2).
2. *A difficulty ladder.* We grade every puzzle by the weakest of five nested deduction engines that solves it without search (unit propagation, single-constraint elimination, one- and two-level lookahead, search), alongside solver statistics and minimal-core explanation sizes, and relate the ladder to LLM accuracy for three model sizes (E3).
3. *Fair but hard.* We evolve puzzles with LLM-proposed edits under the constraint that they stay unique and solvable at L1–L3, with an LLM solver's mistakes as fitness (E4), and let an LLM invent new genres that the pipeline compiles and certifies (E5).

All puzzles, certificates, ladder levels and explanations are released as a benchmark that can be regenerated freshly at any time.

Our main findings are mostly about limits. Encoders

made no errors on plain template text, and the errors we did find came from ambiguous wording and from meaning drift introduced by a paraphrasing model, more than from misreading; dual encoding caught independent misreadings but not errors shared by both encoders, which the uniqueness check caught instead. LLM-written puzzles were nearly always valid but easy and highly redundant, and some Sudoku copied a well-known published puzzle. The ladder tracks solver effort closely, but current models solve almost all fair zebra, Killer and Futoshiki puzzles regardless of level, so it says little about LLM accuracy on these sizes; 9×9 Sudoku is the exception. Evolution against the smallest model found no fair zebra puzzle it failed.

2 Methods

2.1 Puzzles as finite-domain constraint problems

Every puzzle in this study is a finite-domain constraint satisfaction problem with three parts: variables with small domains, *rules* of the genre that every instance shares, and *clues* that define one instance and can be removed. A rule or clue is either an all-different constraint or a table constraint, given by a predicate over the values of a small scope of variables. We compile the problem to CNF with the direct encoding: one Boolean $x_{v,a}$ per variable v and value a, exactly-one clauses per variable, pairwise at-most-one clauses for all-different (plus at-least-one clauses when the scope is a permutation, such as a Sudoku row), and one clause per forbidden tuple of a table constraint. Uniqueness is checked by solving, adding a clause that blocks the solution and solving again; the second call must be unsatisfiable [14, 30]. We use MiniSat 2.2 through PySAT [4, 9] and repeat every certificate with an independent model in OR-tools CP-SAT [20]. Both solvers agreed on every puzzle we certified.

Families. We use four families. *Zebra* puzzles have N houses and K categories with N values each (default N=5, K=4); clues are generated from 11 relation types (same person, different people, house position, not at a position, directly left of, next to, not next to, somewhere left of, exactly one house between, at an end, either–or) and rendered into English by templates with several phrasings each. Each zebra clue carries a canonical expression in the position DSL described below, which is the ground truth for its meaning. *Sudoku* is the classic 9×9 game. *Killer Sudoku* is played on 6×6 grids with 2×3 boxes; clues are cages (sum, digits distinct) and, rarely, givens. *Futoshiki* is played on 5×5 Latin squares with inequality clues between neighbours and optional givens.

Certified generation. For each family we draw a random solution, build a pool of clues that are true in it, reach uniqueness, and minimise. For zebra puzzles we start from the empty clue set and repeatedly add a pool clue that is violated by the current counter-example solution, which guarantees termination. For the grid families we start from the full pool. Minimisation removes clues one at a time in random order (givens first for Killer and Futoshiki) and keeps a removal whenever the solution stays unique, so every clue of a released puzzle is necessary. All randomness is seeded; every stage writes JSONL checkpoints and is resumable.

2.2 The difficulty ladder

We grade a puzzle by the weakest deduction engine that reaches the full solution without search:

L1 unit propagation on the CNF. In Sudoku terms this covers naked and hidden singles; for clues it covers using a clue once the other side is known.

L2 L1 plus single-constraint elimination: generalised arc consistency on each rule or clue taken on its own, together with the exactly-one constraints of its scope [15, 21]. This covers eliminations licensed by one clue, naked and hidden subsets inside one row or category, and cage combinations.

L3 L2 plus failed-literal probing: place a candidate value, run unit propagation, and eliminate the candidate if this yields a contradiction [8, 11].

L4 L2 plus two-level lookahead: place a candidate, then run unit propagation and failed-literal probing inside that hypothesis.

L5 none of the above suffices; the solver must search.

All rules are sound and monotone, so each engine has a unique fixpoint, and the level can be computed in one pass that enables stronger rules only when weaker ones are exhausted. We call a puzzle *fair* if it is unique and graded L1–L3. Unit tests check the ladder on hand-built instances that sit exactly at each of the five levels, on the classic Einstein puzzle, and on a published Sudoku with known answer.

Solver-side metrics. For each puzzle we record (i) the fraction of variables fixed after the L1 and L2 fixpoints (a "propagation backbone"; the full backbone of a unique puzzle is trivially every variable), (ii) MiniSat decisions, conflicts and propagations for finding the solution and proving it unique, (iii) for every non-given fact, the size of a minimal set of clues that forces it, computed as a group minimal unsatisfiable subset over clue selectors by deletion [1, 12], and (iv) clue redundancy, the fraction of clues whose individual removal keeps the solution unique.

2.3 LLM calls

No API key was available in the environment where this study was run, and open-weight models could not be downloaded. Every LLM call was therefore executed as a fresh Claude Code subagent with no shared context, selected by model alias (Haiku, Sonnet, Opus); a probe call confirmed the

alias resolved to `claude-haiku-5-5`, and the other aliases are documented as `claude-sonnet-5-5` and `claude-opus-5-5`. A pipeline stage writes one task file per call; the worker is instructed to read that file, reason without tools, and write one JSON output file. The harness reports the total token count and the number of tool calls of each subagent, which we log. Workers are told not to run code. The harness counts three tool events for a compliant call (read, write, hand-back); 16 of 240 calls logged more. We matched 239 calls to their transcripts and listed every tool call: none ran code or read another file; 11 made one extra call to a session-status or messaging tool without puzzle content or edited their own output file, and 5 wrote their output file in several steps. Several items share a call (10 puzzles per encoding or generation call, 4–5 per solving call), so items within a call are not independent of each other, but different encoders or solvers of the same item never share a call. Model reasoning settings were the harness defaults and are not under our control. The same task files can be run against the Messages API with one function in the repository.

2.4 E1: dual-encoding verification

An encoder receives the puzzle text and a short description of a position DSL in which `pos('tea')` is the house number of a value and each clue is one Python Boolean expression over `pos`, integers, comparisons, `Abs`, `And`, `Or`, `Not` and `Implies`. The prompt gives three worked examples (same person, fixed position, next to) and leaves every other relation to the model. The same expression can be evaluated concretely (to build table constraints) or symbolically in z3 [3]. Two encodings A and B are equivalent when $\mathit{base} \wedge (A \oplus B)$ is unsatisfiable, where *base* states the domains and the per-category all-different rules. The same test against the generator's own expressions gives a ground-truth check for single encodings, at the level of the whole puzzle and of each clue.

We encoded 299 zebra puzzles in three text conditions: 100 with plain templates; 100 with "hard" templates that use indirect references ("the left-hand neighbour of the Dane keeps the cat"), counting from the right, negated orderings, conditionals, "neither ... nor" and "between"; and 99 whose plain-template clues Sonnet rewrote into natural prose under an instruction to keep the meaning exactly (one of 100 paraphrases was misaligned and excluded). Each puzzle was encoded three times: twice by Sonnet in independent calls with different batch compositions (A and B), and once by Haiku (C). For every clue-level disagreement with the ground truth we assigned an error category automatically, by testing equivalence with a list of plausible misreadings, and then read every case by hand.

2.5 E2: validity of LLM-written puzzles

Sonnet wrote 40 zebra, 20 Sudoku, 20 Killer and 20 Futoshiki puzzles together with their intended solutions, in batches of ten, under an instruction that each must have exactly one solution and be solvable without guessing. Zebra puzzles were dual-encoded as in E1 and kept only when the two encodings were equivalent. A puzzle is *valid* if it parses, has exactly one solution, that solution is the stated one, and it is fair. The solver loop then (i) returns the solver's feedback (a second solution, the clues the stated solution violates, or a minimal conflicting clue set) to a fresh Sonnet call for repair, and (ii) minimises the puzzle by clue removal. We also measured redundancy and searched for repeated Sudoku layouts.

2.6 E3: ladder, solver metrics and LLM accuracy

From the certified banks (1,500 minimal puzzles per family, plus 8,000 extra Sudoku drawn to look for rare high levels) we sampled up to eight puzzles per family and level. Minimal zebra, Killer and Futoshiki puzzles are almost never L1, so the L1 cells were filled by adding random true clues to minimal L2–L3 puzzles until they reached L1; these are marked as augmented. No minimal Sudoku above L3 occurred among 9,500 generated, and only four L5 Killer puzzles occurred among 1,500. The final set has 116 puzzles. Haiku, Sonnet and Opus each solved every puzzle (4–5 per call, same batches for every model). We score exact solution and cell accuracy (over non-given cells); an unanswered item scores zero. We relate level to solver metrics by Spearman correlation with bootstrap intervals [5], and to accuracy with a logistic regression of exact solution on level, family and model fitted on the family–model cells that are not at ceiling (a pooled fit is separated, since some cells have no errors), with puzzle-level cluster bootstrap for the level coefficient, and a linear mixed model of cell accuracy with a random intercept per puzzle [23].

2.7 E4: evolving fair puzzles against a solver

We ran a $(\mu{+}\lambda)$ evolution of zebra puzzles with $\mu = 8$. Each generation Haiku (the training solver) solves every individual; fitness is one minus the mean cell accuracy over all of an individual's evaluations, so survivors are re-evaluated every generation and lucky scores average out; ties prefer fewer clues. Sonnet proposes two variants per parent as edit operations on the formal clue list (add, remove or replace a clue of a typed relation) after seeing the puzzle, its solution and the solver's mistakes. The pipeline applies valid edits, discards clues that are false in the solution, restores uniqueness with random true clues if needed, and rejects any child that is not fair (L>3). Held-out testing re-solves the initial and final populations with

fresh Haiku calls and with Sonnet and Opus. After the zebra runs (a 5×4 pilot and a 6×6 run) found no solver errors, we ran the same loop on 9×9 Sudoku, where Haiku had failed in E3: the initial population is 8 random minimal L3 Sudoku (as for zebra, the initial population is drawn at L3), Sonnet proposes edits to the givens (add, remove or move a given, using the solution), children are repaired to uniqueness by adding givens from the solution and must stay at L1–L3.

2.8 E5: LLM-invented genres

Sonnet was asked to invent six new genres and to write each as Python code against a small API: a grid size, a function that adds global rules (all-different or table constraints over cells) and a function that, given a solution grid, lists candidate clues true of it with a checking predicate and a text. The pipeline compiles the code in a restricted namespace, draws random solutions of the rules with a SAT solver, builds a minimal unique clue set from the candidate clues, and grades the result. We generated twelve instances per genre.

3 Results

3.1 Certified banks

Every puzzle in the four banks was certified unique by MiniSat and, independently, by CP-SAT, and every clue of every puzzle is necessary. Table 1 shows the ladder levels. Minimal puzzles rarely sit at L1: removing every redundant clue removes the easy routes, so the weakest engine that suffices is usually single-constraint elimination (L2) or failed-literal probing (L3). Minimal Sudoku is the exception, because with 9 digits per unit a few givens already create hidden singles; across 9,500 random minimal Sudoku none needed more than L3. The ladder separates the families: Killer reaches L4 in a fifth of the puzzles and L5 in four, zebra reaches L4 in about one in nine, and 5×5 Futoshiki never needs more than L3. Puzzles generated from the hard text templates have a similar level distribution to plain ones, because the templates change the wording, not the constraint.

3.2 E1: how often is a single encoding wrong?

Figure 1 shows one puzzle end to end: the text, the two Sonnet encodings (they differ in form on three clues and are proved equivalent), and the chain of forced facts with the minimal clue set behind each. Table 2 and Figure 2 give the error rates.

On plain template text no encoder made an error: 0 of 3×1,369 clue encodings were wrong. On hard templates Sonnet A produced no wrong puzzle, Sonnet B 2 and Haiku 6 of 100 (6% [2, 11]). On paraphrased text all three encoders were wrong on 7–8 of 99 puzzles (Sonnet A 7% [3, 12]). These rates are low and their intervals are wide; the more useful result is the audit of what went wrong (Section 3.2).

Detection. The dual check flagged every puzzle on which exactly one encoder of a pair was wrong. It missed the errors that both encoders shared: on paraphrased text the two Sonnet encodings disagreed on no puzzle at all, although both were wrong on the same 7. Each of those wrong encodings, however, made the encoded puzzle lose its unique solution, so the uniqueness check rejected it. Across all 299 puzzles and all three encoders, no wrong encoding passed both checks (Figure 2b). Uniqueness works as a second detector because a minimal puzzle has no slack: changing the meaning of one clue almost always either removes the intended solution or admits a second one.

Audit. We read every clue-level error by hand (`results/e1/audit.md`). There were four groups. (i) *An ambiguous template of our own* (45 clue encodings per encoder): "At least one house separates X and Y" was generated for "X and Y are not neighbours" ($|p_X - p_Y| \neq 1$, which allows the same house), and all three encoders read it as $|p_X - p_Y| \geq 2$. Their reading is the natural one; the template is wrong. Because the clue is still true of the solution, no answer changed, and neither check can see the error; only the formal ground truth revealed it. (ii) *Counting from the right*: "the fourth house from the right" encoded as house 4 (Haiku 5 clues, Sonnet B 1). (iii) *Direction flips*: "the left-hand neighbour of the German drinks juice" encoded with the German on the left (2 clues). (iv) *Paraphrase drift*: Sonnet, asked to keep the meaning exactly, rewrote "exactly one house between X and Y" as "two houses separate X and Y" in 7 puzzles. The encoders then encoded what the text said. This is an error of the text, not of the encoding, and dual encoding by construction cannot catch it.

3.3 E2: LLM-written puzzles without and with the loop

Sonnet's puzzles were almost always valid (Table 3): 98 of 100 parsed, had a unique solution equal to the stated one, and were fair. One Sudoku had a stated solution different from its true unique solution, and one Futoshiki had several solutions; one round of solver feedback repaired both (100 of 100). The zebra puzzles' two encodings never disagreed.

The puzzles were, however, easy and redundant (Figure 3). All 20 Sudoku had exactly 30 givens and were solved by unit propagation alone; half of the Killer puzzles were L1. On average 0.73 of a Sudoku's givens, 0.84 of a Killer puzzle's clues and 0.79 of a Futoshiki puzzle's clues could each be removed without losing uniqueness; for zebra puzzles the figure was 0.29. Minimising the LLM puzzles with the solver raised their level (Sudoku: 5 of 20 reached L3; zebra: 14 of 40). The 20 Sudoku used only 8 distinct layouts of givens; 7 used the layout of the example puzzle on the

Table 1: Ladder levels of the certified banks (minimal puzzles). All puzzles were certified unique by two solvers.

Family	n	L1	L2	L3	L4	L5	clues (mean)
Zebra	1,500	0	360	967	173	0	13.6
Sudoku	1,500	738	214	548	0	0	24.3
Killer	1,500	0	582	600	314	4	8.4
Futoshiki	1,500	0	619	881	0	0	10.1
Sudoku (extra)	8,000	3859	1110	3031	0	0	24.3
Zebra (hard text)	300	0	57	198	45	0	13.2

Table 2: E1. Puzzles with a wrong encoding (in brackets: wrong clue encodings) for each encoder, and puzzles on which a pair of encoders disagrees (in brackets: puzzles with at least one wrong encoder in the pair that the dual check flagged / all such puzzles).

		wrong single encoding			dual check: disagree (caught/with error)		
Text	n	Sonnet A	Sonnet B	Haiku C	A–B	A–C	B–C
Plain templates	100	0 (0)	0 (0)	0 (0)	0 (0/0)	0 (0/0)	0 (0/0)
Hard templates	100	0 (15)	2 (17)	6 (21)	2 (2/2)	6 (6/6)	6 (6/7)
LLM paraphrase	99	7 (7)	7 (7)	8 (8)	0 (0/7)	1 (1/8)	1 (1/8)

English Wikipedia Sudoku page and 1 reproduced that puzzle exactly. LLM-written puzzles are thus valid but not a good source of hard or novel instances, and copying is a real risk for benchmarks built this way.

3.4 E3: the ladder, solver effort and LLM accuracy

Ladder versus solver metrics. Within every family the ladder level tracks solver effort and explanation size (Figure 4, Table 4). The Spearman correlation of level with MiniSat conflicts was 0.95 (zebra), 0.74 (Sudoku), 0.91 (Killer) and 0.86 (Futoshiki); with the share of variables fixed by unit propagation it was between -0.84 and -0.91. The level is not a proxy for the number of clues: in Sudoku the correlation with the number of givens was 0.13, and in the other families harder puzzles had *fewer* clues. Mean explanation size rises with level, so harder puzzles are also those in which each fact depends on more clues at once.

Ladder versus LLM accuracy. Table 5 and Figure 5 give the exact-solution rates. Every model solved every zebra puzzle at every level, including the 8 L4 puzzles. Opus solved all 116 puzzles; Sonnet missed one L5 Killer puzzle (with 35 of 36 cells right). Only Haiku made more than isolated errors, and mostly on Sudoku (62 % exact). Haiku's Sudoku accuracy fell from 6 of 8 at L1 and L2 to 3 of 8 at L3, but this drop needs care: 6 of its 24 Sudoku items were left unanswered, 5 of them L3, and 5 of the 6 came from one call that gave up on its whole batch of five. Among the items Haiku did answer, it solved 6/7 at L1, 6/8 at L2 and 3/3 at L3. Whether giving up is itself a sign of difficulty cannot be separated here from the batch it happened in.

Formally, on the family–model cells that are not at ceiling (Sudoku, Killer and Futoshiki for Haiku and Sonnet; $n = 168$), the logistic slope of exact solution on level was -0.45 (95% cluster-bootstrap interval -1.35 to 0.24; $p = 0.16$). A linear mixed model of cell accuracy over all 348 attempts gave a level slope of -0.010 per level (SE 0.008, $p = 0.18$). Adding the level to a model with MiniSat conflicts, explanation size and number of clues lowered the AIC from 88.4 to 87.0, so the ladder carries some information that raw solver effort does not, but the evidence is weak. The honest summary is that for current models these puzzle sizes are mostly too easy for the ladder to predict accuracy: it predicts solver effort well and LLM failure poorly, because there are few failures to predict.

3.5 E4: evolving fair puzzles against a solver

Zebra. A pilot on 5×4 zebra puzzles and the main run on 6×6 zebra puzzles both ended without a single training-solver error. Over 3 evaluated generations the 6×6 run produced 33 fair individuals (3 at L2, 30 at L3) from 25 accepted of 32 proposed edits, and Haiku solved every one of 49 evaluations exactly; the pilot (22 individuals, 30 evaluations) was the same. With no error, fitness is zero everywhere and selection has nothing to act on (Figure 6a). Fresh calls of all three models also solved every initial and final individual (Figure 6b). Fair zebra puzzles of this size are therefore not hard for Haiku, and we stopped the zebra runs.

Sudoku. PENDING

3.6 E5: genres invented by an LLM

All 6 genres that Sonnet wrote compiled in the sandbox, and the pipeline built and certified 12 minimal unique instances of each (Table 6). Most genres are Latin squares with a new clue type: parity and sums of neighbours, sums of mirrored cells, cells a knight's move apart that must differ, or bans on neighbouring

a Puzzle (natural language)

```
hobby: cooking, tennis, chess, painting, golf
job: pilot, farmer, baker, lawyer, teacher
color: yellow, white, purple, red, green
nationality: Spaniard, Brit, German, Swede, Dane

 1. The person who enjoys tennis and the farmer are
    different people.
 2. The person who enjoys painting lives directly to the
    left of the person who enjoys golf.
 3. The Swede and the person in the white house live two
    houses apart.
 4. The baker lives in the house just left of the one
    where the Swede lives.
 5. The teacher and the occupant of the yellow house are
    neighbours.
 6. The Dane lives in the green house.
 7. The cooking enthusiast lives in the red house.
 8. The baker lives directly to the left of the person
    who enjoys chess.
 9. Whoever lives in the white house lives in a lower-
    numbered house than the person who enjoys chess.
10. The lawyer and the teacher live two houses apart.
11. The teacher lives immediately to the right of the
    person who enjoys cooking.
12. The occupant of the red house and the Swede live in
    adjacent houses.
13. The Spaniard and the Swede live in adjacent houses.
14. The German lives in house 3.
```

b Two independent encodings

A and B are separate Sonnet calls; z3 proves (A xor B) unsatisfiable, i.e. A and B are equivalent.

```
 1 A: pos('tennis') != pos('farmer')
   B: (identical up to case)
 2 A: pos('golf') == pos('painting') + 1
   B: pos('painting') + 1 == pos('golf')
 3 A: Abs(pos('Swede') - pos('white')) == 2
   B: (identical up to case)
 4 A: pos('baker') == pos('Swede') - 1
   B: pos('baker') + 1 == pos('swede')
 5 A: Abs(pos('teacher') - pos('yellow')) == 1
   B: (identical up to case)
 6 A: pos('Dane') == pos('green')
   B: (identical up to case)
 7 A: pos('cooking') == pos('red')
   B: (identical up to case)
 8 A: pos('chess') == pos('baker') + 1
   B: pos('baker') + 1 == pos('chess')
 9 A: pos('white') < pos('chess')
   B: (identical up to case)
10 A: Abs(pos('lawyer') - pos('teacher')) == 2
   B: (identical up to case)
11 A: pos('teacher') == pos('cooking') + 1
   B: (identical up to case)
12 A: Abs(pos('red') - pos('Swede')) == 1
   B: (identical up to case)
13 A: Abs(pos('Spaniard') - pos('Swede')) == 1
   B: (identical up to case)
14 A: pos('German') == 3
   B: (identical up to case)
```

c Forced deductions (ladder L3)

facts in the order the ladder engine derives them; L = highest rule needed so far; {..} = minimal set of clues that forces the fact

1	german = house 3	L1	{14}
2	cooking = house 3	L3	{4,5,7,8,9,11,12,14}
3	tennis = house 5	L3	{2,3,4,5,7,8,9,11,12,14}
4	chess = house 4	L3	{3,4,5,7,8,9,11,12,14}
5	painting = house 1	L3	{2,3,4,5,7,8,9,11,12,14}
6	golf = house 2	L3	{2,3,4,5,7,8,9,11,12,14}
7	pilot = house 5	L3	{1,2,3,4,5,7,8,9,10,11,12,14}
8	farmer = house 1	L3	{1,2,3,4,5,7,8,9,10,11,12,14}
9	baker = house 3	L3	{3,4,5,7,8,9,11,12,14}
10	lawyer = house 2	L3	{4,5,7,8,9,10,11,12,14}
11	teacher = house 4	L3	{4,5,7,8,9,11,12,14}
12	yellow = house 5	L3	{4,5,7,8,9,11,12,14}
13	white = house 2	L3	{3,4,5,7,8,9,11,12,14}
14	purple = house 4	L3	{3,4,5,6,7,8,9,11,12,14}
15	red = house 3	L3	{4,5,7,8,9,11,12,14}
16	green = house 1	L3	{3,4,5,6,7,8,9,11,12,14}
17	spaniard = house 5	L3	{3,4,5,7,8,9,11,12,13,14}
18	brit = house 2	L3	{3,4,5,6,7,8,9,11,13,14}
19	swede = house 4	L3	{3,4,5,7,8,9,11,12,14}
20	dane = house 1	L3	{3,4,5,6,7,8,9,11,13,14}

Figure 1: One zebra puzzle through the pipeline. (a) The text given to the encoders. (b) The two Sonnet encodings in the position DSL; three clues are written differently and z3 proves $A \oplus B$ unsatisfiable. (c) Every fact of the solution in the order the ladder derives it, the strongest rule needed so far, and a minimal set of clues that forces it. The puzzle is L3: after one unit-propagation step every further fact needs failed-literal probing, and most explanations need 8–13 of the 14 clues.

Table 3: E2. Validity, level and redundancy of 100 puzzles written by Sonnet. "Valid" = parses, unique, matches the stated solution and fair (L1–L3). Levels are given as L1/L2/L3 counts before and after solver minimisation.

Family	n	unique	as stated	valid	after repair	levels (LLM)	levels (min.)	redundancy	clues
Zebra	40	40	40	40	40	1/37/2	1/25/14	0.29	16.3 → 14.1
Sudoku	20	20	19	19	20	20/0/0	6/9/5	0.73	30.0 → 22.2
Killer	20	20	20	20	20	10/10/0	0/17/3	0.84	18.6 → 10.1
Futoshiki	20	19	19	19	20	1/18/0	0/13/6	0.79	15.0 → 9.1

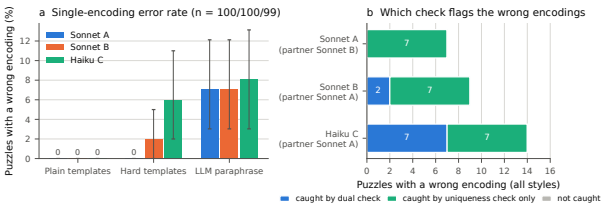


Figure 2: E1. (a) Share of puzzles with a wrong single encoding, with 95% bootstrap intervals. (b) For each encoder, how its wrong encodings were detected: by disagreement with its partner encoding, or only because the wrongly encoded puzzle no longer had a unique solution. No wrong encoding escaped both checks.

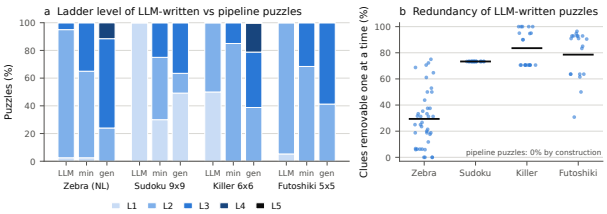


Figure 3: E2. (a) Ladder levels of the LLM-written puzzles, of the same puzzles after solver minimisation, and of the pipeline's own bank. (b) Fraction of clues that can each be removed without losing uniqueness; the pipeline's puzzles have none by construction.

digits two apart. Their certified instances are mostly L3. The genre we find most interesting is *Median Line Latin* (Figure 7e): a 5×5 Latin square whose clues either give the sum of two or three neighbouring cells or state that the middle of three consecutive cells has a value strictly between the other two. Eight of its 12 instances need two-level lookahead (L4) with about 7 clues, the deepest of all genres. The "median" clue is weak on its own (it rules out only the two extreme values of the middle cell) but combines with the Latin rules into long chains, which is what the

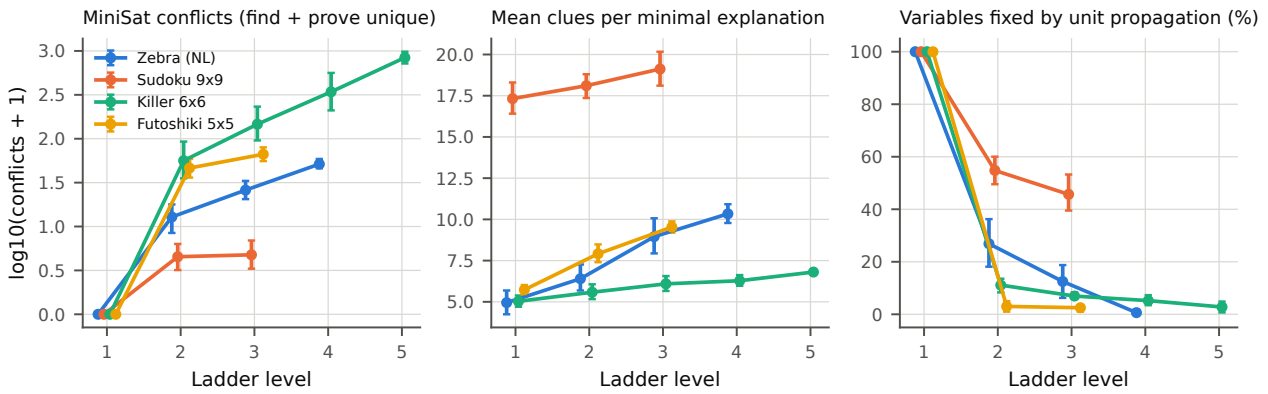


Figure 4: E3. Solver-side metrics by ladder level for the 116 selected puzzles (means with 95% bootstrap intervals). L1 cells of zebra, Killer and Futoshiki contain augmented puzzles.

Table 4: E3. Spearman correlation of ladder level with solver metrics, with 95% bootstrap intervals.

Metric	Zebra	Sudoku	Killer	Futoshiki
MiniSat conflicts	0.95 [0.87, 0.98]	0.74 [0.41, 0.93]	0.91 [0.80, 0.97]	0.86 [0.64, 0.97]
MiniSat decisions	0.92 [0.81, 0.97]	0.73 [0.39, 0.93]	0.92 [0.82, 0.97]	0.85 [0.62, 0.96]
mean MUS size	0.88 [0.78, 0.93]	0.45 [0.04, 0.77]	0.71 [0.49, 0.85]	0.91 [0.80, 0.95]
fixed by L1	-0.91 [-0.97, -0.80]	-0.84 [-0.96, -0.62]	-0.87 [-0.96, -0.72]	-0.77 [-0.94, -0.45]
probing steps	0.84 [0.70, 0.90]	0.84 [0.70, 0.89]	0.67 [0.39, 0.84]	0.84 [0.70, 0.89]
number of clues	-0.73 [-0.87, -0.48]	0.13 [-0.33, 0.57]	-0.77 [-0.91, -0.52]	-0.55 [-0.79, -0.07]

Table 5: E3. Exact solutions (%) with 95% bootstrap intervals, all levels pooled. Unanswered items count as wrong (Haiku: 6 Sudoku and 2 Futoshiki items).

Family	Haiku	Sonnet	Opus
Zebra	100 [100, 100]	100 [100, 100]	100 [100, 100]
Sudoku	62 [42, 79]	100 [100, 100]	100 [100, 100]
Killer	94 [86, 100]	97 [92, 100]	100 [100, 100]
Futoshiki	88 [75, 100]	100 [100, 100]	100 [100, 100]

Table 6: E5. Genres invented by Sonnet: certified instances and their ladder levels.

Genre	certified	L1	L2	L3	L4	L5	clues
Parity Pairs Latin	12/12	0	4	8	0	0	4.7
Median Line Latin	12/12	0	0	4	8	0	7.2
Shifted Rows	12/12	0	0	5	7	0	12.7
Mirror Sum Latin	12/12	0	2	10	0	0	3.4
Knight Strangers	12/12	0	0	12	0	0	3.9
No Gaps of Two	12/12	0	0	9	3	0	3.0

ladder detects. All three models nevertheless solved all four L4 instances of this genre that we gave them (12 of 12 attempts).

4 Discussion

What the solver loop buys. The loop turns a model's claim ("this puzzle has one answer") into a certificate, but the certificate covers the encoding, not the text. Our results suggest three practical rules. First, the translation from text to constraints is reliable when the text is regular: on template text no encoder made any error. Second, dual encoding catches independent misreadings, which are what a second model run is good at, but not errors that two runs share, and two runs of the same model share a great deal: the two Sonnet encoders agreed on every paraphrased puzzle, including the seven they both got wrong. Third, uniqueness is a strong and cheap second check for minimal puzzles, because a minimal puzzle has no slack: a misread clue almost always removes the intended solution or lets in another. The combination caught every wrong encoding we observed. What it cannot catch is an error that leaves the puzzle unique with the same answer, such as our own ambiguous template; such errors do not change the answer but do mean the text says something other than the certificate, which matters if the puzzle is used to teach or to explain.

LLM-written puzzles. Sonnet's puzzles were valid far more often than we expected, and one round of solver feedback fixed the rest. The problem is not validity but quality: they are easy, padded with redundant clues, and in Sudoku they reuse a famous layout. A solver that minimises and grades the puzzle fixes the first two; the third is a warning for anyone who builds evaluation sets by asking a model for puzzles.

The ladder. Five nested propagation engines give a level that is cheap to compute, interpretable in solving terms, and closely related to solver effort and to the size of minimal explanations in every family. It does not, at these sizes, predict whether current LLMs succeed, because they almost always do. The exception is the

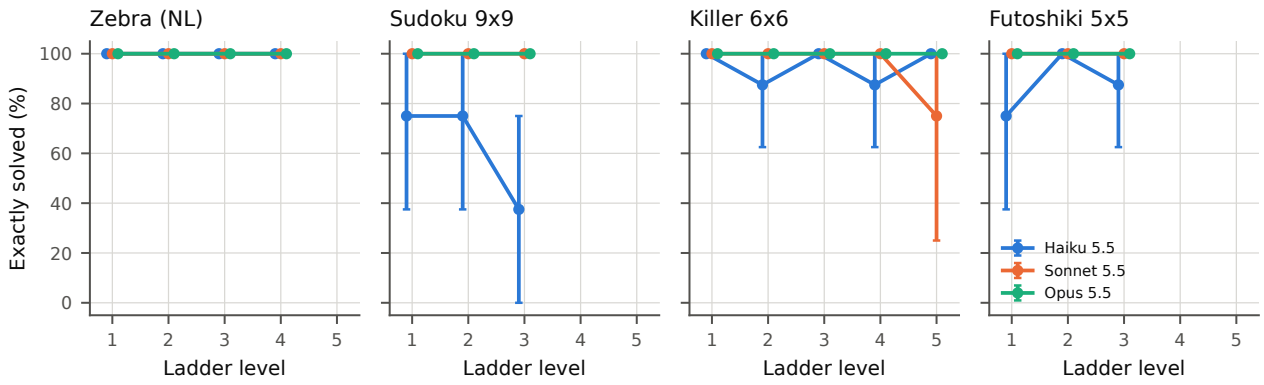


Figure 5: E3. Exact-solution rate by ladder level, family and model (95% bootstrap intervals; 8 puzzles per cell except Killer L5 with 4). Sudoku has no L4–L5 cells because none occurred among 9,500 generated puzzles; Futoshiki 5×5 has none above L3.

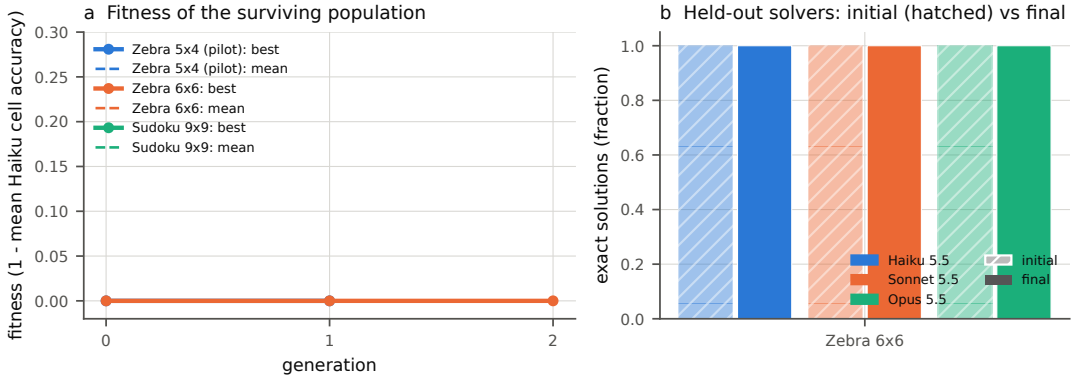


Figure 6: E4. (a) Best and mean fitness of the surviving population per generation (fitness = 1 − mean Haiku cell accuracy over all evaluations of an individual). (b) Exact-solution rate of fresh solver calls on the initial (hatched) and final populations.

smallest model on 9×9 Sudoku, where failures, and especially give-ups, concentrate in the higher levels, though our data cannot separate level from batch effects there. A ladder whose top levels are rare in random generation (no Sudoku above L3 in 9,500) is also a ladder whose top levels must be searched for, which is what E4 attempts.

Fair but hard. We did not find fair zebra puzzles that Haiku fails. This is a null result at our scale (two runs, at most three generations, 8 individuals) and it does not show that such puzzles do not exist; but the space we searched, all unique L1–L3 puzzles of size up to 6×6, was solved without error in 49 of 49 attempts. Larger instances, other genres, or fitness based on solving cost rather than errors may behave differently.

5 Limitations

- *How the models were called.* No API key was available, so every call was a fresh Claude Code subagent with no shared context (Section 2.3). Temperature and reasoning settings were not under our control, model versions are identified by alias, and the workers had a general system prompt we did not write. The audit of tool calls (239 of 240 calls matched to transcripts) found no code execution; 11 calls made one extra status or messaging call with no puzzle content (one edited its own output file), and 5 wrote their output in several steps. Calls with extra tool use were kept.
- *Scale.* The study is small: 299 puzzles in E1, 100 in E2, 116 in E3, three evolution runs. Rates near zero have wide intervals, and E3 has 8 puzzles per cell (4 for Killer L5).
- *Batching.* Several items share a call. Items in a call are not independent, and a call that gives up affects all its items (one Haiku Sudoku call left five items unanswered, which we score as wrong).
- *Augmented L1 puzzles.* L1 cells of three families were filled by adding redundant clues to minimal puzzles, so they differ from the other cells in clue count and redundancy.
- *Ground truth in E1* comes from our generator; one of its templates was ambiguous, which we report as a finding rather than an encoder error.
- *Cost* is estimated from total token counts with a conservative split between input and output and list

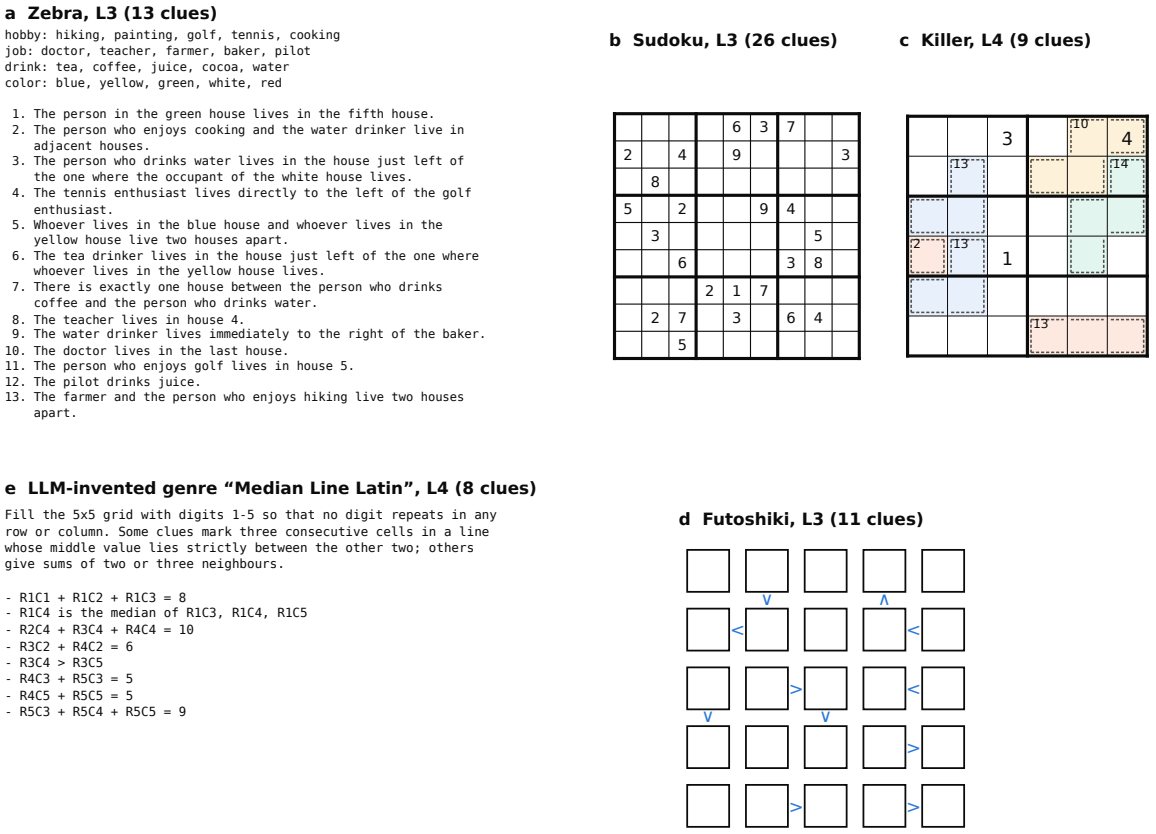


Figure 7: Gallery of certified puzzles from the benchmark: (a) zebra, (b) Sudoku, (c) Killer (cages with sums), (d) Futoshiki, (e) the LLM-invented genre Median Line Latin. Levels are ladder levels; every puzzle is minimal and unique.

prices without caching; real costs would be lower.

- *Ladder.* The ladder is defined on one CNF encoding; a different encoding (for example with auxiliary variables) would change what unit propagation can do, and humans' notion of difficulty is not modelled.

Data and code availability

All code, configuration, prompts, task files, model outputs, intermediate results and figures are in the project repository (`https://github.com/sarabehnamian/Puzzle-generation-with-guaranteed-uniqueness`). The benchmark (`benchmark/`) contains the certified puzzles with their solutions, ladder levels, solver metrics and minimal-core explanations, with a datasheet [6]. Every number in this paper is written to `paper/numbers.tex` by `src/make_numbers.py` from saved results. The model calls used 81 Haiku, 130 Sonnet and 29 Opus calls with 16.8 million tokens in total; at list prices without caching this would cost about $45 through the API (an upper-bound estimate).

Acknowledgements

The experiments, analysis code and a first draft of this text were produced with the help of Claude Code (Anthropic); the author is responsible for the content.

References

[1] Bart Bogaerts, Emilio Gamba, and Tias Guns. A framework for step-wise explaining how to solve constraint satisfaction problems. *Artificial Intelligence*, 300:103550, 2021.

[2] Cameron Browne and Frederic Maire. Evolutionary game design. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(1):1–16, 2010.

[3] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008.

[4] Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In *Theory and Applications of Satisfiability Testing (SAT 2003)*, volume 2919 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2004.

[5] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

[6] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

[7] Shahriar Golchin and Mihai Surdeanu. Time travel in LLMs: Tracing data contamination in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Marijn J. H. Heule and Hans van Maaren. Look-ahead based SAT solvers. In *Handbook of Satisfiability*, pages 155–184. IOS Press, 2009.

[9] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: A Python toolkit for prototyping with SAT oracles. In *Theory and Applications of Satisfiability Testing (SAT 2018)*, volume 10929 of *Lecture Notes in Computer Science*, pages 428–437. Springer, 2018.

[10] Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, et al. Dynabench: Rethinking benchmarking in NLP. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4110–4124, 2021.

[11] Chu Min Li and Anbulagan. Look-ahead versus look-back for satisfiability problems. In *Principles and Practice of Constraint Programming (CP 1997)*, volume 1330 of *Lecture Notes in Computer Science*, pages 341–355. Springer, 1997.

[12] Mark H. Liffiton and Karem A. Sakallah. Algorithms for computing minimal unsatisfiable subsets of constraints. *Journal of Automated Reasoning*, 40(1):1–33, 2008.

[13] Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. ZebraLogic: On the scaling limits of LLMs for logical reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[14] Inês Lynce and Joël Ouaknine. Sudoku as a SAT problem. In *Proceedings of the 9th International Symposium on Artificial Intelligence and Mathematics (ISAIM)*, 2006.

[15] Alan K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8(1):99–118, 1977.

[16] Gary McGuire, Bastian Tugemann, and Gilles Civario. There is no 16-clue Sudoku: Solving the Sudoku minimum number of clues problem via hitting set enumeration. *Experimental Mathematics*, 23(2):190–217, 2014.

[17] Theo X. Olausson, Alex Gu, Ben Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger P. Levy. LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[18] Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[19] Radek Pelánek. Difficulty rating of Sudoku puzzles: An overview and evaluation. arXiv:1403.7373, 2014.

[20] Laurent Perron and Frédéric Didier. CP-SAT, part of OR-Tools. Google, https://developers.google.com/optimization, 2024.

[21] Jean-Charles Régin. A filtering algorithm for constraints of difference in CSPs. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI)*, pages 362–367, 1994.

[22] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[23] Skipper Seabold and Josef Perktold. Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010.

[24] Noor Shaker, Julian Togelius, and Mark J. Nelson. *Procedural Content Generation in Games*. Springer, 2016.

[25] Helmut Simonis. Sudoku as a constraint problem. In *CP Workshop on Modeling and Reformulating Constraint Satisfaction Problems*, pages 13–27, 2005.

[26] Adam M. Smith and Michael Mateas. Answer set programming for procedural content generation: A design space approach. *IEEE Transactions on Computational Intelligence and AI in Games*, 3 (3):187–200, 2011.

[27] Nemika Tyagi, Mihir Parmar, Mohith Kulkarni, Aswin RRV, Nisarg Patel, Mutsumi Nakamura, Arindam Mitra, and Chitta Baral. Step-by-step reasoning to solve grid puzzles: Where do LLMs falter? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[28] Tjark Weber. A SAT-based Sudoku solver. In *LPAR-12, The 12th International Conference on Logic for Programming, Artificial Intelligence and Reasoning, Short Paper Proceedings*, pages 11–15, 2005.

[29] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[30] Takayuki Yato and Takahiro Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E86-A(5): 1052–1060, 2003.

[31] Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. SatLM: Satisfiability-aided language models using declarative prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.